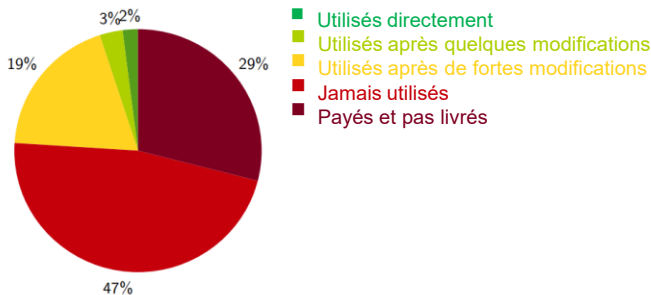


Introduction aux Génie Logiciel

Crise du logiciel

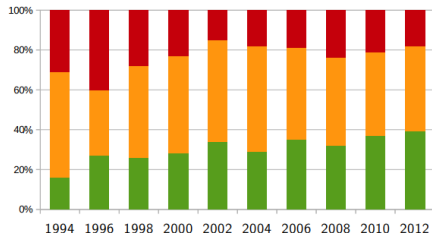
Étude du Department of Defense des États-Unis sur les logiciels produits dans le cadre de 9 gros projets militaires (Étude du DoD, 1995).



Jarzombek, Stanley J., The 5th annual JAWS S3 proceedings, 1999

Crise de logiciel

Étude du Standish group: Enquête sur des milliers de projets, de toutes tailles et de tous secteurs

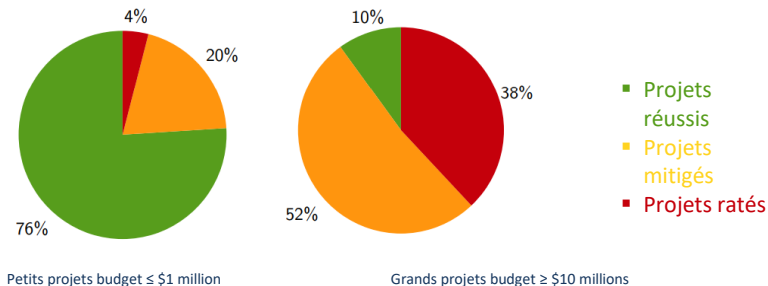


Standish group, Chaos Manifesto 2013 - Think Big, Act Small,
2013

- **Projets réussis** : achevés dans les délais et pour le budget impartis, avec toutes les fonctionnalités demandées
- **Projets mitigés** : achevés et opérationnels, mais livrés hors délais, hors budget ou sans toutes les fonctionnalités demandées
- **Projets ratés** : abandonnés avant la fin ou livrés mais jamais utilisés

Crise du logiciel

Petits vs grands projets (Standish group, Chaos Manifesto 2013 - Think Big, Act Small, 2013)



Le taux de succès décroît avec la taille des projets et la taille des entreprises.

Raisons de la faible qualité des logiciels

Tâche complexe :

- Taille et complexité des logiciels

- Taille des équipes de conception/développement

Manque de méthodes et de rigueur :

- Manque de méthodes de conception

- Négligence et manque de méthodes et d'outils des phases de validation/vérification

Mauvaise compréhension des besoins :

- Négligence de la phase d'analyse des besoins du client

- Manque d'implication du client dans le processus

Historiques

Bugs célèbres:

1962: Destruction en vol de la sonde spatiale **Mariner 1** peu de temps après son envol

- ❑ Coût : 18,5 millions de dollars
- ❑ Mission: survol de la planète Vénus
- ❑ Cause: Erreur de transcription manuelle d'un symbole mathématique dans la spécification

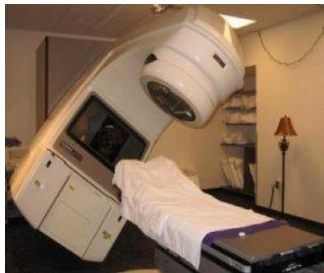


Historiques

Bugs célèbres:

1985/1987: Problème du **Therac-25**, surdosage en radiothérapie « appareil d'irradiation thérapeutique »

- ❑ Au moins 5 morts par dose massive de radiations
- ❑ Cause: des fautes de conception dans le matériel et le logiciel



Historiques



Bugs célèbres:

1994: le bug de la division du **Pentium**, une erreur était introduite en raison d'une faute de conception de l'algorithme de division flottante

1996: Ariane 5, explosion du premier vol d'Arian 5 due à une erreur de débordement lors de la conversion d'un nombre flottant 64 bits vers un entier 16 bits.

Coût 500 M \$. (Le bug informatique le plus couteux de l'histoire)



2000 : Le bug de, dysfonctionnements lorsque les dates sont postérieures au 31 décembre 1999



2011: PlayStation Network, des millions de données personnelles et bancaires piratées, pertes financières de plusieurs milliards de dollars, a cause des vulnérabilités du réseau

2014: Outil de chiffrement, Open SSL, 500000 serveurs web concernés par la faille. vulnérabilité permettant de lire une portion de la mémoire d'un serveur distant



Définition: Un logiciel est un ensemble de programmes informatiques, de données et de documentation associée conçus pour réaliser des tâches spécifiques sur un ordinateur ou un système informatique.

Les logiciels sont développés en utilisant **des langages de programmation** et sont exécutés par un ordinateur pour effectuer des opérations telles que le traitement de l'information, le stockage des données, ou le contrôle du matériel.

Définition: Le génie Logiciel est une discipline qui concerne l'ensemble des méthodes, des techniques et des outils dédiés à la **conception**, au **développement** et à la **maintenance** des systèmes informatiques.

Objectif: L'objectif du génie logiciel est de développer des logiciels de haute qualité, fiables et efficaces tout en respectant les contraintes de coûts et de délais.

Objectifs de GL

Objectif GL (Critère CQFD):

Le GL se préoccupe des procédés de fabrication de logiciel de façon à s'assurer que les quatre critères suivant soit satisfait:

- **Coût (C):** les coûts reste dans les limites prévu au départ.
- **Qualité (Q):** La qualité correspond au contrat du service initial.
- **Fonctionnalité (F):** Le système qui est fabriqué répond aux besoins des utilisateurs.
- **Délai (D):** Les délais reste dans les limites prévu au départ.

Qualités attendues d'un logiciel (1/2)

Des facteurs de qualité du logiciel :

La validité: aptitude d'un produit logiciel à réaliser exactement les tâches définies par sa spécification.

La robustesse: aptitude d'un logiciel à fonctionner même dans des conditions anormales.

L'extensibilité: Facilité d'adaptation d'un logiciel aux changements de spécification.

Réutilisabilité: aptitude d'un logiciel à être réutilisé en tout ou en partie pour des nouvelles applications.

La compatibilité: aptitude d'un logiciel à pouvoir être combiné avec d'autres.

Qualités attendues d'un logiciel (2/2)

D'autres facteurs de qualité du logiciel sont moins cruciaux:

L'efficacité: bonne utilisation des ressources du matériel.

La portabilité: capacité à pouvoir s'adapter à différents environnements matériel ou logiciel.

Vérifiabilité: facilité de préparation des procédures de recette et de certification (test, ...).

L'intégrité: aptitude des logiciels à protéger leurs différents composants contre accès et des modifications non autorisés.

Facilité d'utilisation: facilité avec lesquelles les utilisateurs d'un logiciel peuvent apprendre comment l'utiliser, comment le faire fonctionner, comment préparer les données, mais aussi comment interpréter les résultats et les effets en cas d'erreur.

Cycle de vie d'un logiciel (1/9)

Le cycle de vie du logiciel modélise l'enchaînement des différentes activités du processus technique de développement du logiciel.

Une activité comprend: des tâches, des contraintes, des ressources, une façon d'être réalisée.

Les grandes activités sont:

- Analyse des besoins
- spécification globale
- Conception architectural et détaillé
- Programmation
- Validation et vérification
- Livraison et maintenance

Cycle de vie d'un logiciel (2/9)

1.1. Analyse des besoins:

But: Eviter de développer un logiciel non adéquat. On va étudier le **domaine d'application** ainsi que l'état **actuel** et **futur** de l'environnement du système afin d'en déterminer:

Les frontières, **Le Rôle**, Les **ressources disponibles** et requises, Les contraintes d'utilisation et performance ... etc.



- Experts du domaine d'application
- Futurs utilisateurs du système



- Entretien
- Questionnaire
- Observation de l'existence
- Etude de situation similaire

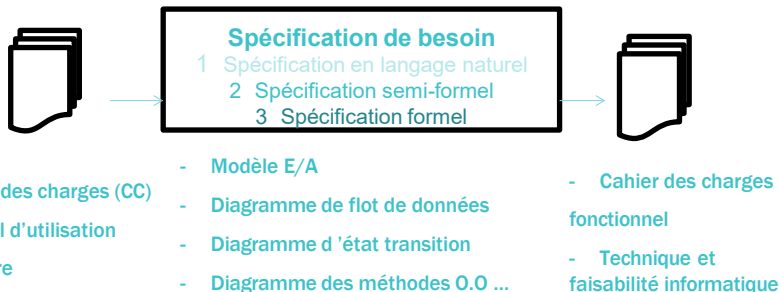


- Cahier des charges (CC)
- Manuel d'utilisation préliminaire

Cycle de vie d'un logiciel (3/9)

1.2. Spécification des besoins:

But: Etablir une première description du futur système. pour produire une description de ce que doit faire le système mais sans préciser comment il le fait



Cycle de vie d'un logiciel (4/9)

1.3. Conception:

But: Enrichir la description du logiciel de détails d'implémentation afin d'aboutir à une description très proche d'un programme (décrire **le comment**).

- ✓ **La conception architecturale** (ou conception globale) a pour but de décomposer le logiciel en composants plus simples, définis par leurs interfaces et leurs fonctions (les services qu'ils rendent).
- ✓ **La conception détaillée** fournit pour chaque composant une description de la manière dont les fonctions ou les services sont réalisés : algorithmes, représentation des données.

Cycle de vie d'un logiciel (5/9)

1.3. Conception:



- Cahier des charges fonctionnel
- Technique et faisabilité informatique

- Conception fonctionnelle et conception orienté objet
- Pour chaque composant, écrire les algorithmes

- Dossier de conception: (Architecture et Algorithme)

Cycle de vie d'un logiciel (6/9)

1.4. Programmation:

Implantation de la solution conçue

- ✓ Choix de l'environnement de développement, du/des langage(s) de programmation, de normes de développement...



- Dossier de conception:
(Architecture et Algorithme)

- Code source
- Manuel final d'utilisateur
- Documentations

Cycle de vie d'un logiciel (7/9)

1.6. Validation et vérification:

La validation a pour but de répondre à la délicate question : a-t-on décrit le bon système , celui qui répond à l'attente des utilisateurs.

La vérification répond à la question : le développement est-il correct par rapport à la spécification globale ? Ce qui consiste à s'assurer que les descriptions successives et le logiciel lui-même satisfont la spécification.

Cycle de vie d'un logiciel (8/9)

1.7. Maintenance:

Il s'agit d'apporter des modifications à un logiciel existant. C'est la phase la plus coûteuse (70% du coût total)

2. Documentation

Objectif : Traçabilité du projet

Pour l'équipe :

- Regrouper et structurer les décisions prises
- Faire référence pour les décisions futures
- Garantir la cohérence entre les modèles et le produit

Pour le client :

- Donner une vision claire de l'état d'avancement du projet

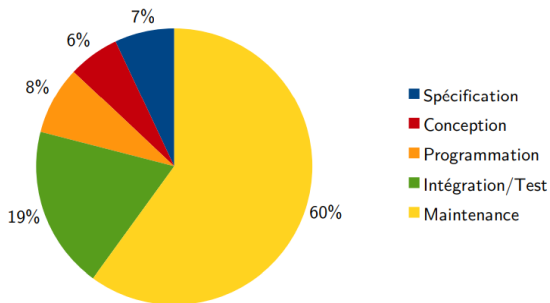
Base commune de référence :

- Personne quittant le projet : pas de perte d'informations
- Personne rejoignant le projet : intégration rapide

Le coût de développement d'un logiciel

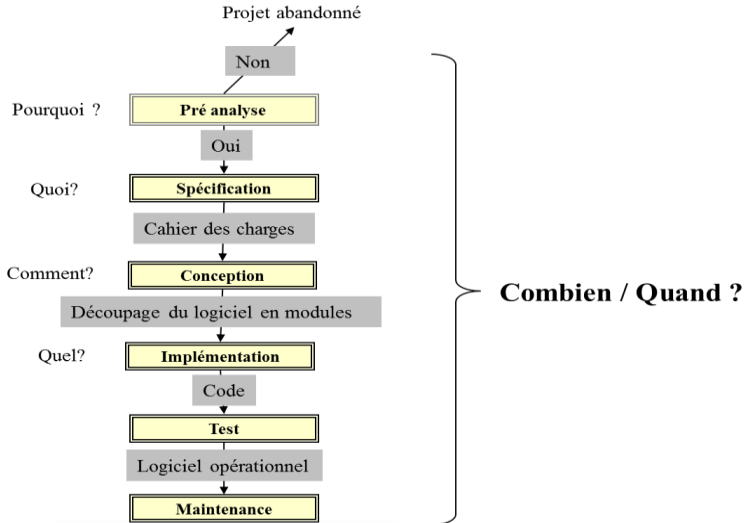
- ❑ dépend de la taille et de la complexité du projet.
- ❑ varie en fonction des fonctionnalités requises, des technologies utilisées
- ❑ varie en fonction du nombre de développeurs impliqués.

Répartition de l'effort



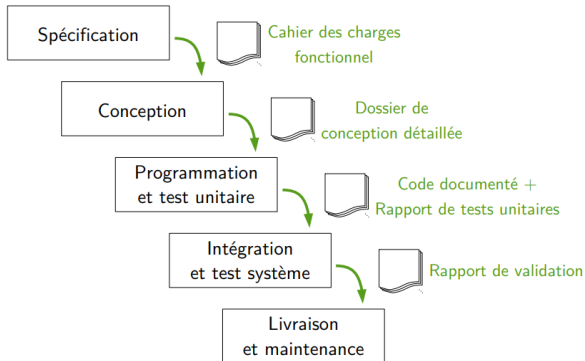
La maintenance absorbe une très grande partie des efforts de développement

Cycle de vie d'un logiciel



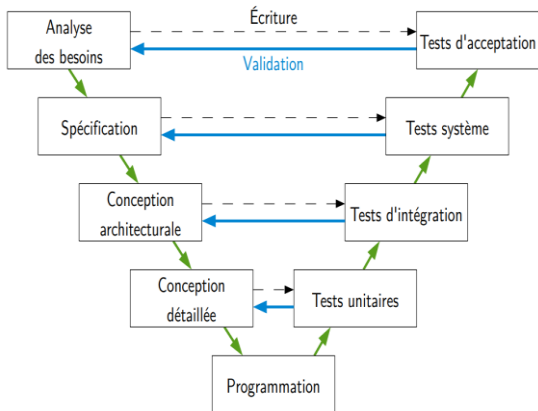
Modèle en Cascade

1. Les phases du modèle en cascade sont séquentielles et linéaires, ce qui rend le processus de développement facile à suivre.
2. Chaque phase nécessite une documentation détaillée, ce qui peut être bénéfique pour les projets où une documentation exhaustive est nécessaire.
3. convient aux projets où les exigences sont clairement définies à l'avance et où les changements sont peu probables.



Modèle en V

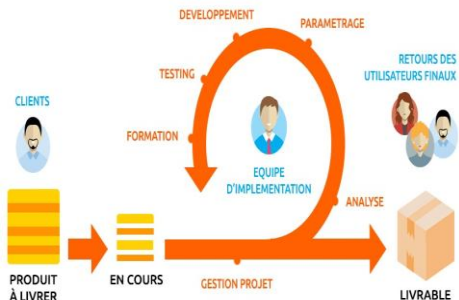
Le modèle en V met donc l'accent sur la vérification et la validation à chaque étape du processus de développement, ce qui permet de détecter et de corriger les erreurs dès qu'elles se produisent, réduisant ainsi les risques et les coûts associés aux corrections tardives.



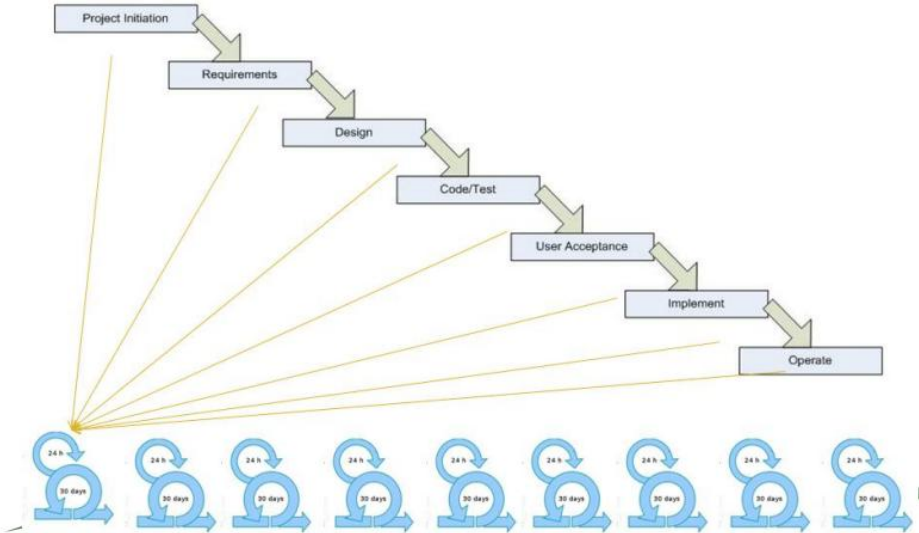
Modèle Agile

Le modèle Agile est une approche de développement logiciel itérative et incrémentale qui met l'accent sur la flexibilité, la collaboration et la livraison rapide de fonctionnalités utiles.

Le modèle Agile favorise des cycles de développement courts appelés "itérations" ou "sprints", généralement de quelques semaines à quelques mois, au cours desquels des fonctionnalités sont conçues, développées, testées et livrées.



Modèle Agile



Modèle Agile

Scrum est un processus empirique

